

클래스+연산자오버로딩 실습

- 각 과제를 4개의 폴더에서 각각 구현하여 **10/10(토) 23:00**까지 제출
 - 10/17(토)까지 추가 제출 가능(60% 만점으로 채점)
 - 폴더에 합치지 말고 폴더 전체를 한꺼번에 압축할 것
 - \1.Fraction : Fraction.cpp Fraction.h 과 \2.Matrix : Matrix.cpp Matrix.h
 - 지난 과제 개인피드백과 공통피드백을 모두 확인하고 반영하여 수정할 것
 - 기존 Matrix::input()을 cin에 대한 operator >> 으로 변경할 것
 - Fraction과 Matrix에 대해서는 본인이 작성한 main은 제출하지 말 것.
 - \3.Money : Money.cpp Money.h (main 미포함)
 - \4.DDay : 적절히 파일 분할하여 *.cpp를 컴파일하면 동작하도록 구성
 - 주된 클래스는 DDay.cpp DDay.h로 구성할 것. **main()의 적절한 구조 고려하여 직접 구현하고 포함하여 제출**
 - 필요하다면 common.h와 common.cpp (또는 utils.h utils.cpp)등의 구현을 권장
- 다음 내용을 주요 채점 요소로 함
 - 코딩 스타일 지속적으로 채점. Class 설계 적절성을 매우 중요한 기준으로 평가
 - 함수에서의 적절하고 정확한 const 정의 여부도 매우 중요.
 - 입출력 제외 모든 **operator overload**는 **멤버 함수로 구현**. 입출력은 **friends 함수 사용 금지**. 앞으로의 모든 과제 적용.
 - 표준 **c++ 컴파일러**에서 컴파일 되도록, **Visual Studio**가 아닌 **g++** 으로 컴파일되도록 할 것. 앞으로의 모든 과제 적용.

1.Fraction & 2.Matrix

- 이전 과제의 분수와 행렬 프로그램을 아래 main으로 동작하게 변경하시오.
 - 아래 main에 포함되어 있는 함수만 포함한 별도 main으로 테스트 예정
 - Fraction의 setter는 set()하나만 사용하는 것으로 변경
 - Matrix에서 출력 줄맞춤에 대해 엄격하게 채점. 자리수 다양하게 채점할 예정

```
#include <iostream>
#include "Fraction.h"
using namespace std;

int main()    2/3 + -2/5 = 4/15
{
    Fraction f1, f2(2,-5), f3;

    f1.set(2,3);

    f3 = f1+f2;

    cout << f1 << " + " << f2;
    cout << " = " << f3 << endl;

    return 0;
}
```

```
#include <iostream>
#include "Matrix.h"
using namespace std;

int main()
{
    Matrix m1, m2;

    cin >> m1;    cout << m1;
    cin >> m2;    cout << m2;

    cout << "Transpose " << endl;
    cout << !m1;

    cout << "Add " << endl;
    cout << m1 + m2;

    cout << "Multiply " << endl;
    cout << m1 * m2;
    return 0;
}
```

```
1 2 3 4 5 6 7 8 9
| 1 2 3 |
| 4 5 6 |
| 7 8 9 |
1 -1 0 0 -1 1 -1 1 0
| 1 -1 0 |
| 0 -1 1 |
| -1 1 0 |
Transpose
| 1 4 7 |
| 2 5 8 |
| 3 6 9 |
Add
| 2 1 3 |
| 4 4 7 |
| 6 9 9 |
Multiply
| -2 0 2 |
| -2 -3 5 |
| -2 -6 8 |
```

3.Money

- dollar와 cent에 대한 class Money 에 대해 아래 연산자와 함수들을 구현
 - binary operator +, -, ==, !=, >=, <=, >, < 와, 적절한 생성자와 toString()도 구현
 - 반드시 dollarPart와 centPart를 구분하여 처리할 것. 총 cent 또는 double 로 처리하면 감점 큼.
 - 음수 금액이 정확하게 출력되도록 신경써서 구현할 것
- 오른쪽 코드가 동작하도록 구현

```
30 120
$31.20
-9 10
-$8.90
$31.20 + -$8.90 = $22.30
$31.20 - -$8.90 = $40.10
$31.20 == -$8.90 is false.
$31.20 >= -$8.90 is true.
$31.20 <= -$8.90 is false.
$31.20 > -$8.90 is true.
$31.20 < -$8.90 is false.
```

```
Money m1, m2;

cin >> m1 ; cout << m1 << endl ;
cin >> m2 ; cout << m2.toString() << endl ;

cout << m1 << " + " << m2
    << " = " << m1 + m2 << endl ;

cout << m1 << " - " << m2
    << " = " << m1 - m2 << endl ;

cout << m1 << " == " << m2 << " is ";
if (m1 == m2) cout << "true" << endl;
else cout << "false." << endl;

...
```

4. DDay

- class Day: 년/월/일 정보를 저장(default 생성자는 올해 10월 1일)
- operator overloading을 구현
 - 단항 전위 연산자 ++와 --, 산술연산자 +와 -, 출력 <<
 - Day today, dDay ; dDay = today + 100과 같은 형태
 - operator +와 operator -의 단일 인자는 정수. 결과는 Day형 instance.
 - + 100을 ++를 100번 수행하는 등의 비효율적인 방식으로 구현하면 감점 큼
- 실행 예제와 같이 돌아가게 구성.
 - 날짜를 직접 쓰거나, 전날이나 다음날로 이동하면 현재 날짜가 변경
 - 입력되는 글자수는 최대 8개로 제한하여 char [9] 선언 + 정수 직접 구할 것. string으로 선언하여 stoi쓰면 감점.
 - 날짜가 변경되면, D-day를 변경된 날짜에 맞추어 변경할 것
 - D-day는 <https://search.naver.com/search.naver?query=디데이계산기> 기준으로 계산
 - “+정수”로 표현되는 이후 d-day는 네이버페이지의 “후”
 - “-정수”로 표현되는 이전 d-day는 네이버페이지의 “전”의 결과와 동일하게 나와야 함
 - “<< “로 시작하는 줄은 cout << currentDate ; 로 출력
- class Day 객체를 포함하는 class DDay 를 구현하는 것을 권장
 - 보다 나은 설계에 대해서 고민할 것

```

<< 2026/10/01 [D-day:+0] 2026/10/01
>> Move date{yyyymmdd, Tomorrow(T/t), Yesterday(Y/y)}, Set D-day(+/-int), or Quit(Q/q) : y
<< 2026/09/30 [D-day:+0] 2026/09/30
>> Move date{yyyymmdd, Tomorrow(T/t), Yesterday(Y/y)}, Set D-day(+/-int), or Quit(Q/q) : T
<< 2026/10/01 [D-day:+0] 2026/10/01
>> Move date{yyyymmdd, Tomorrow(T/t), Yesterday(Y/y)}, Set D-day(+/-int), or Quit(Q/q) : +1004
<< 2026/10/01 [D-day:+1004] 2029/07/01
>> Move date{yyyymmdd, Tomorrow(T/t), Yesterday(Y/y)}, Set D-day(+/-int), or Quit(Q/q) : Y
<< 2026/09/30 [D-day:+1004] 2029/06/30
>> Move date{yyyymmdd, Tomorrow(T/t), Yesterday(Y/y)}, Set D-day(+/-int), or Quit(Q/q) : t
<< 2026/10/01 [D-day:+1004] 2029/07/01
>> Move date{yyyymmdd, Tomorrow(T/t), Yesterday(Y/y)}, Set D-day(+/-int), or Quit(Q/q) : 20300229
*** ERROR
<< 2026/10/01 [D-day:+1004] 2029/07/01
>> Move date{yyyymmdd, Tomorrow(T/t), Yesterday(Y/y)}, Set D-day(+/-int), or Quit(Q/q) : 20240229
<< 2024/02/29 [D-day:+1004] 2026/11/29
>> Move date{yyyymmdd, Tomorrow(T/t), Yesterday(Y/y)}, Set D-day(+/-int), or Quit(Q/q) : -8826
<< 2024/02/29 [D-day:-8826] 1999/12/31
>> Move date{yyyymmdd, Tomorrow(T/t), Yesterday(Y/y)}, Set D-day(+/-int), or Quit(Q/q) : t
<< 2024/03/01 [D-day:-8826] 2000/01/01
>> Move date{yyyymmdd, Tomorrow(T/t), Yesterday(Y/y)}, Set D-day(+/-int), or Quit(Q/q) : y
<< 2024/02/29 [D-day:-8826] 1999/12/31
>> Move date{yyyymmdd, Tomorrow(T/t), Yesterday(Y/y)}, Set D-day(+/-int), or Quit(Q/q) : a
*** ERROR
<< 2024/02/29 [D-day:-8826] 1999/12/31
>> Move date{yyyymmdd, Tomorrow(T/t), Yesterday(Y/y)}, Set D-day(+/-int), or Quit(Q/q) : q
=== END ===

```